

Rapport projet Snake

Benjamin BAUDOT

Valentin VINCENT

Mohan GROS-DESORMEAUX

Yu Yu CHEN

18 novembre 2023

Table des matières

1	Introduction	1
2	Partie Terminal	1
2.1	Déroulement du jeu	1
2.2	Sauvegarde	2
2.3	Menu	4
3	Partie SDL (Interface Graphique)	4
3.1	Les Fenêtres	4
3.2	Le Jeu	5
4	Outils utilisés	5
5	Répartition des tâches et du temps	6
6	Difficultés rencontrées	6

1 Introduction

Nous avons pour projet de réaliser un jeu Snake en langage de programmation C. Le but du jeu est de contrôler un serpent qui va se nourrir de bonus au cours de la partie. Le serpent ne doit cependant pas se prendre un mur ou un obstacle ou encore sa propre queue. Son score est comptabilisé selon le nombre de bonus collectés.

Le but de notre projet était de réaliser un émulateur du jeu Snake dans un terminal ainsi qu'une interface graphique. Cependant nous avons ajouté un nombre limité de déplacements avant que le bonus ne change de position sur la carte ainsi qu'un système de sauvegarde multi-utilisateurs.

2 Partie Terminal

2.1 Déroulement du jeu

Pour la partie terminal il nous allons créer une carte qui sera extraite à partir d'un fichier, créer un serpent à partir des notions étudiées en cours et créer une base de données qui contiendra les données du joueur, son score ainsi que son nombre de pas. Pour récupérer la carte, nous avons procédé à la lecture ligne par ligne car on sait que dans la première ligne du fichier contenant la carte nous connaissons le nombre de lignes et le nombre de colonnes puis nous allons lire le reste (donc la carte) de la même façon (ligne par ligne). Celles-ci seront ensuite stockées «

imprimées » avec les obstacles qui seront ici représentés par des « # » (croisillon) dans une matrice créée à partir d'une allocation dynamique nous permettant ainsi d'obtenir les coordonnées de chaque case et savoir s'il s'agit d'un mur ou d'un obstacle.

Le serpent sera une liste chaînée composée par 1 ou plusieurs nœuds. Chaque nœud contiendra une coordonnée sur la carte qui devront être uniques sachant que leurs coordonnées ne devront pas correspondre aux coordonnées d'un obstacle (« # ») ou aux coordonnées d'une partie de son corps sinon la partie est terminée et le joueur a perdu.

Nous allons ensuite calculer la génération suivante du serpent. Pour la calculer nous allons utiliser la direction que le joueur aura choisie en entrée puis à partir de la direction choisie nous allons récupérer les coordonnées de la tête du serpent pour en calculer de nouvelles qui seront la nouvelle position de la tête du serpent. Par exemple :

- Si la direction pointée est en bas ("I") alors on va enlever 1 à l'ordonnée de la tête du serpent
- Si la direction pointée est en haut ("O") alors on va ajouter 1 à l'ordonnée de la tête du serpent
- Si la direction pointée est à gauche ("K") alors on va enlever 1 à l'abscisse de la tête du serpent
- Si la direction pointée est à droite ("M") alors on va ajouter 1 à l'abscisse de la tête du serpent

Dans le cas où le joueur entre une direction opposée à celle du serpent, celle-ci ne sera pas prise en compte.

Dans un cas normal, c'est-à-dire si la longueur de notre serpent est supérieure à 1 alors on pousse les coordonnées à partir de la tête soit le nœud précédent deviendra le nœud suivant. Pour cela, nous allons récupérer les coordonnées du nœud suivant et remplacer les coordonnées de notre nœud actuel par celles du suivant.

Dans le cas où nous avons seulement, la tête celle-ci uniquement sera traitée. Avant chaque actualisation, on teste les coordonnées de la tête avec celle de fruit si elles sont identiques alors le serpent mange le fruit et génère une nouvelle tête à la place de fruit en effaçant le fruit sur le terrain et l'ancienne tête devient le corps de serpent puis on passe au cas normal. On appelle coordonnée la position entre i et j avec i le nombre de ligne et j le nombre de colonne dans la matrice qui contiendra la carte. Si la tête de serpent a une coordonnée égale $[0,i]$ ou $[n,i]$ ou $[j,0]$ ou $[j,m]$ avec un entier naturel $j \geq 0$ et $\leq n$ et un entier naturel $i \geq 0$ et $\leq m$ alors le jeu arrête et lance la demande de nouvelle partie à l'utilisateur. Celui aura alors le choix entre recommencer une nouvelle partie (option : oui) ou bien quitter le jeu (option : non).

Lorsque le serpent collecte un nouveau bonus, les coordonnées de sa tête prendront les coordonnées du fruit puis nous insérons un nouveau nœud à l'arrière donc à la première position soit l'indice 1 de notre liste chaînée (serpent) du serpent avec valeur z . Si le joueur dépasse par contre les 10 déplacements, son nombre de déplacements est remis à zéro et le fruit est supprimé puis un nouveau fruit est généré.

2.2 Sauvegarde

Pour la partie sauvegarde, qui fut l'une des parties les plus compliquées à réaliser nous avons testé plusieurs méthodes de sauvegardes :

La première consistait en la sauvegarde de la structure du joueur qui contiendrait :

- son identifiant ;
- son score actuel ;
- son meilleure score ;
- son nombre de pas ;
- la carte du jeu ;
- le serpent lui-même

Or le problème de cette méthode est que les adresses qui contiennent les valeurs du serpent et de la carte seront supprimées à la fin du programme et que si nous voulions conserver ces données cela aurait probablement causé des fuites mémoires. Donc lorsque le joueur va vouloir récupérer ses données, celles-ci seront perdues car nous avons stocké les adresses dans notre structure et non les valeurs.

- La deuxième méthode reprend le principe de la première mais au lieu de sauvegarder la carte du jeu et le serpent nous avons choisi de faire deux chaînes de caractères avec une qui contiendrait les données de chaque noeud du serpent (coordonnées et valeurs séparées par des “;” entre noeud et “,” entre éléments du noeuds) et les données de la carte dans l’autre. Nous aurions aussi choisi de rajouter deux éléments dans la structure du joueur, le nombre de lignes ainsi que le nombre de colonnes qui seront ensuite récupérés pour reproduire la carte lors du prochain lancement du programme puis la lecture du serpent avec les conditions de “.” et “;” pour récupérer les coordonnées et valeurs.
- La dernière méthode qui est celle que nous avons utilisée consiste à sauvegarder la structure du joueur dans un fichier qui prendra son nom que nous allons concaténer avec l’extension “.txt” puis sauvegarder dans un répertoire player. La structure du joueur va contenir :
 - son score ;
 - son nombre de pas ;
 - chemin du fichier contenant la carte ;
 - la dernière direction du joueur
 - l’abscisse du fruit
 - l’ordonnée du fruit
 - la valeur du fruit

Les coordonnées de chaque nœud du serpent seront, elles, stockées dans un fichier qui portera également le nom du joueur que l’on va concaténer lui aussi avec l’extension “.txt” et qui sera stocké dans un autre répertoire nommé sauvegarde. Chaque coordonnées seront stockées lignes à lignes dans le fichier. À noter qu’ici on ne prend pas la valeur de chaque nœud et nous expliquerons pourquoi par la suite.

Nous avons également ajouté une option qui supprime la sauvegarde du joueur lorsque le joueur a perdu afin d’éviter qu’il puisse “tricher” et revenir à sa sauvegarde autant de fois qu’il veut.

Pour la récupération des données du joueur nous allons simplement lire dans les 2 répertoires (sauvegarde et player dans le répertoire data) les 2 fichiers qui ont pour nom celui des joueurs (avec l’extension “.txt”) et récupérer les infos qu’ils contiennent. On récupère d’abord la structure du joueur dans un fichier et à partir des fonctions que nous avons créées nous allons pouvoir relancer la création de la carte mais cette fois avec le nom de la carte qui est dans la structure du joueur. Ensuite nous allons ouvrir le second fichier dans lequel nous récupérerons les coordonnées du serpent qui sont stockés ligne à ligne. Exemple de contenu dans un fichier contenant le serpent du joueur dont le nom du joueur sera “joueur” :

colonne
x
y
x1
y1
x2
y2
...
xn
yn

Une fois les données des 2 fichiers extraites, nous allons les convertir de façon à pouvoir les réutiliser ensuite dans nos fonctions initiales soit reconstruire le serpent, reconstruire la carte (nous utiliserons la même fonction que pour la construire), récupérer le nombre de pas pour le fruit et permettre de limiter les déplacements, et récupérer son score. Le nom nous servira simplement à rechercher si les fichiers contenant les données existent.

2.3 Menu

Pour terminer nous avons construit un menu dans lequel le joueur devra d'abord s'identifier afin de récupérer ses fichiers puis nous avons mis 3 options :

- Nouvelle partie
- Charger une partie (afficher seulement si le joueur avait sauvegarder et quitter la partie en cours)
- Quitter le jeu

Nous avons aussi rajouté un menu lorsque le joueur appuie sur la touche "q" de son clavier soit lorsqu'il veut quitter la partie :

- Sauvegarder et quitter
- Réinitialiser à zéro
- Nouvelle carte
- Quitter

3 Partie SDL (Interface Graphique)

Pour la partie interface graphique, il nous a fallu repenser énormément de choses concernant le fonctionnement, l'affichage, le chargement, etc... du serpent et ce n'était pas joué d'avance car nous devions commencer de zéro dans notre partie car il nous fallait apprendre la bibliothèque qui nous servirait à donner un visage et une couleur à notre serpent. En plus de cela pour apprendre l'utilisation de cette bibliothèque, nous n'avons pas eu de cours sur comment est ce que SDL fonctionnait et comment s'en servir. Nous avons donc suivi ce plan afin d'initier le projet :

- Nous avons dû installer et configurer SDL, SDL_image pour les images et SDL_ttf pour le texte
- Après avoir installé les différentes bibliothèques, il a fallu configurer le tout sur vscode pour que ça fonctionne correctement.
- Quand tout cela était fait, nous sommes passés aux fenêtres

3.1 Les Fenêtres

Les fenêtres avec SDL sont des fenêtres qui que l'on peut ouvrir avec les dimensions que l'on veut, les dimensions étant en pixels. Puis quand une fenêtre est créée, elle ne ressemble qu'à un carré ou rectangle sans rien dedans, avec un fond noir ou coloré car on n'y a pas encore ajouté de background (fond d'écran) qu'il faut ajouter ensuite à l'aide de la bibliothèque SDL_ttf. Ce qui a été réalisé plutôt rapidement car l'ouverture de l'image et sa configuration en fond d'écran se faisait au début de nos fonctions.

L'ajout du texte se faisait également de la même manière, à l'importation des ressources (images, skins etc...) sauf que pour pouvoir écrire quelque chose, il nous fallait avoir des fontes disponibles d'où l'importation de SDL_ttf et le téléchargement de fontes.

On a également ajouté des boutons qui exécutent du code quand on clique dessus, dans notre cas, ils ouvrent différentes fenêtres.

Vers la fin du temps imparti pour réaliser le projet, nous avons rajouté de la musique à l'aide de la bibliothèque mixer de SDL. Quand les initialisations de bases étaient terminées, nous sommes passés à la partie jeu.

1. fenetre_acueil : Cette fonction crée la fenêtre d'accueil du jeu. Elle initialise la fenêtre et le rendu, charge et joue de la musique et des effets sonores, crée et affiche du texte (comme le titre du jeu "SNAKE"), et gère des boutons interactifs comme "Play", "Credits", et "Quitter". Elle gère les interactions de l'utilisateur avec ces boutons et ferme la fenêtre si nécessaire.
2. partie : Cette fonction est similaire à fenetre_acueil, mais elle est dédiée à l'écran de sélection du jeu où l'utilisateur peut choisir de démarrer une nouvelle partie, charger une partie existante, ou retourner au menu principal.
3. nMap : Cette fonction permet à l'utilisateur de choisir une carte pour le jeu. Elle affiche différentes cartes avec des informations comme le nom et l'image de la carte. L'utilisateur peut interagir avec ces cartes et sélectionner celle qu'il souhaite utiliser pour jouer.
4. pauseWindow : Cette fonction crée une fenêtre de pause pendant le jeu. Elle affiche le score actuel et offre des options pour continuer le jeu ou quitter. Selon le choix de l'utilisateur, la fonction retourne une valeur pour contrôler le flux du jeu.

5. `gameWindow` : Cette fonction est le cœur du jeu. Elle initialise la fenêtre du jeu, charge et affiche des textures comme le fond, la tête du serpent, le corps du serpent, et les bonus. Elle gère la logique du jeu, y compris le déplacement du serpent, la collecte de bonus, et détecte les collisions. Elle joue également de la musique pendant le jeu. Si le serpent heurte un obstacle, la fonction envoie l'utilisateur à l'écran de fin de partie.
6. `lostWindow` : Cette fonction s'active lorsque le joueur perd la partie. Elle affiche un écran indiquant que le joueur a perdu, montre le score final, et offre des options pour rejouer ou retourner au menu principal.

Chacune de ces fonctions gère différents aspects du jeu, de l'interface utilisateur à la logique du jeu, en passant par la gestion des ressources audiovisuelles.

3.2 Le Jeu

Pour pouvoir faire un jeu 2d où le serpent se déplace toujours à des endroits prédéfinis, la fenêtre de jeu a été divisée en x carré de pixels, chaque fenêtre de jeu faisait donc $l \times x$ pixels de long et $h \times x$ pixels de hauteur. Ce qui nous crée un sorte de grille invisible qui définit les déplacements de notre personnage.

Quand la fenêtre se fait créer, on crée également la tête de notre snake qui fait une taille de x car on veut qu'il se déplace uniquement dans notre grille créée précédemment et que l'on place au centre de notre fenêtre. Notre tête est une structure qui contient ses coordonnées que l'on peut modifier et un nom qui était prévu pour la sauvegarde.

On lui a ensuite assigné les touches lui permettant de se déplacer donc : `o` (pour aller vers le haut), `k` (pour aller à gauche), `l` (pour aller en bas) et `m` (pour se diriger vers la droite) qui permettaient de déplacer la tête dans les directions souhaitées. Le déplacement ajoutait ou enlevait x pixels aux coordonnées x et y de la tête selon la touche, mais chaque "pas" se faisait manuellement. Alors on a créé une variable qui prend la dernière direction choisie et crée une condition dans notre boucle de jeu qui, quand aucune touche n'est pressée durant le passage dans la boucle, fait que le serpent continue de suivre la dernière direction enregistrée. Pour empêcher les demi-tours directs qui pourraient poser problème, on a créé une condition pour que selon la direction vers laquelle on se dirige, aller à l'opposé directement ne soit pas possible. Par exemple, on ne peut pas aller vers le bas si on allait vers le haut précédemment ou aller vers la droite si on se dirigeait vers la gauche etc...

Ensuite, on a créé le corps du serpent qui était également une structure composée :

- D'une liste de taille indéfinie de têtes de serpent.
- De la taille actuelle du serpent.

Le corps se crée et s'allonge et se supprime grâce à 5 fonctions :

- `createSnake` qui crée un tableau dynamique dont la taille est donnée en entrée par `initialCapacity` en lui allouant de la mémoire et renvoie un pointeur vers le début du tableau.
- `resizeSnake(GridSquare* list, int newCapacity)` qui permet de redimensionner le tableau à la taille `newCapacity`.
- `createBodyPart` qui ajoute une nouvelle partie au corps. Elle fonctionne comme ceci
 1. Si la taille actuelle du tableau est égale à sa capacité maximale, le tableau est redimensionné pour avoir plus d'espace.
 2. Ensuite, un ID unique est attribué à la nouvelle partie du corps.
 3. Les coordonnées x et y sont définies pour la nouvelle partie du corps.
 4. La taille actuelle du tableau est incrémentée pour refléter l'ajout de la nouvelle partie du corps.

La fonction gère automatiquement la redimension du tableau si nécessaire.

- `addToSnake` qui définit les coordonnées de la nouvelle partie du corps qui vient d'être ajoutée.
- `freeSnake` qui libère la mémoire allouée au tableau dynamique.

Le jeu s'arrête quand le serpent tente de sortir des coordonnées de la fenêtre, quand il mange son corps ou quand il décide de quitter l'écran de pause sans reprendre la partie.

On a également créé un bonus qui fait une taille x et qui apparaît à des endroits aléatoires sur la carte. Si il entre en collision avec la tête du serpent, on augmente le score du joueur et on le fait réapparaître à un nouvel endroit aléatoire sur la carte. Par contre, si le serpent fait un certain nombre de déplacements sans entrer en contact avec le même bonus, on baisse le score du joueur si ce même score est supérieur à zéro et on fait apparaître le bonus. La création du bonus se fait dans l'initialisation de la fenêtre mais ses déplacements se font à l'aide de fonctions.

Le score augmente de 15 par bonus récupéré et se réduit de 5 par bonus manqué. `o`

4 Outils utilisés

Pour ce projet, nous avons utilisé plusieurs outils :

- GitHub pour partager les dossiers
- Whats App pour communiquer par messagerie
- Discord pour organiser des réunions
- GDB pour les fuites mémoires
- Le site make8bitart.com pour les différents skins du serpent
- DALL-E pour générer les différentes images de fond ainsi que les maps
- ChatGPT pour les problèmes rencontrés sur lesquels nous sommes restés bloqué trop longtemps

5 Répartition des tâches et du temps

mohan	valentin	benjamin	yuyu
menu + interface graphique	interface graphique	sauvegarde + menu + carte + test	serpent + carte + bonus

Le projet nous aura pour chacun pris plusieurs jours de réalisation. Chacun a évolué grâce à ce projet et appris de nouvelles notions.

Pour réaliser les différentes étapes du jeu, il nous a fallu créer plusieurs fenêtres individuelles, chacune possédant ses propres caractéristiques et pour ce faire, chaque fenêtre unique possède sa propre fonction ce qui va impliquer un problème que lon verra plus tard.

Ce qu'il faut savoir c'est qu'à chaque ouverture de fenêtre, il faut faire des initialisations et des tests d'erreurs pour éviter toute fuite de donnée Pareil pour la fermeture, il faut faire plein de libérations de d'espaces occupés par tout ce qu'on avait chargé à l'ouverture. Pour éviter de devoir tout réécrire à chaque fenêtre, on a créé plusieurs fonctions qui se chargent d'initialiser et de faire les tests pour nous.

6 Difficultés rencontrées

Les principales difficultés rencontrées sont donc :

- Le manque de certains modules pour la détection de saisit en c linux c'est-à-dire que le serpent peut avancer sans devoir attendre la fin de la saisie d'un caractère par le joueur. (Windows : kbhit()). Pour régler ce problème nous avons donc choisi de créer notre propre fonction kbhit() qui aura des fonctionnalités similaires à celles de la fonction du module.
- Problème au niveau du stockage lors de la lecture et l'écriture de structures de type pointeur qui contient plusieurs autres structures de type pointeur à l'intérieur d'un fichier. Pour cela, nous avons changé de méthode et nous n'avons mis aucune structure de type pointeur à l'intérieur du fichier.
- Nous avons aussi eu des difficultés par rapport aux fuites de mémoires.
- l'installation de SDL n'était pas une mince affaire car, il fallait faire toute une configuration pour les chemins que nous n'avons pas réussi. Donc nous avons forcé l'ouverture de SDL et ses bibliothèques pour faire fonctionner nos affaires.
- pour la musique et les effets sonores, nous avons rencontré une grosse difficulté qui fait que nous ne pouvons pas jouer 2 sons en parallèle excepté à l'ouverture de la fenêtre sur sdl donc nous avons donc décidé de mettre uniquement des effets sonores à l'ouverture de la fenêtre.
- la sauvegarde n'a pas été réussie car nous ne parvenions pas à enregistrer le serpent dans un fichier txt ou un fichier binaire.